

SEM LAB EXAM

AI-ASISSTED CODING

NAME:MD.SANIYA

ROLL NO:2403A510E7

BATCH:05

SUBSET:07

Q1:

TASK1: Review the given login controller and identify all possible security flaws.

List each vulnerability, explain why it is a problem, how an attacker could exploit it, and provide recommended fixes.

CODE:

```
// insecure-login.js (example to analyze)
const express = require('express');
const mysql = require('mysql');
const router = express.Router();

const conn = mysql.createConnection({host:'localhost', user:'root', password:'password', database:'lms_db'});

router.post('/login', (req, res) => {
  const email = req.body.email;
  const password = req.body.password;

  // insecure: string concatenation -> SQL injection risk
  const q = "SELECT id, name, password FROM users WHERE email = '" + email + "'";
  conn.query(q, (err, results) => {
    if (err) return res.status(500).send('db error');
    if (results.length === 0) return res.status(401).send('Invalid credentials');

    const user = results[0];

    // insecure: comparing plaintext password
    if (user.password === password) {
      // insecure: returning user id and name directly
      res.json({ id: user.id, name: user.name });
    } else {
      res.status(401).send('Invalid credentials');
    }
  });
});

module.exports = router;
```

OUTPUT:

```
{
  "id": 1,
  "name": "John Doe"
}
```

OBSERVATION:

Credentials are safely kept outside code.

Easy to update without editing code.

More secure for projects.

TASK -2:

PROMPT:

Can you review my login controller and identify the security flaws, and then rewrite the code to make it more readable?" CODE:

```

Click to add a breakpoint 1.js (rewritten clean version)
2 const express = require('express');
3 const mysql = require('mysql2/promise');
4 const bcrypt = require('bcrypt');
5 const router = express.Router();
6
7 // Use environment variables - no hardcoded credentials
8 const pool = mysql.createPool({
9   host: process.env.DB_HOST || 'localhost',
10  user: process.env.DB_USER || 'root',
11  password: process.env.DB_PASS || 'password',
12  database: process.env.DB_NAME || 'lms_db'
13 });
14
15 router.post('/login', async (req, res) => {
16   const { email, password } = req.body;
17
18   try {
19     // secure: prepared statement
20     const [rows] = await pool.execute(
21       'SELECT id, name, password_hash FROM users WHERE email = ?',
22       [email]
23     );
24
25     if (rows.length === 0) {
26       return res.status(401).json({ error: 'Invalid credentials' });
27     }
28
29     const user = rows[0];
30
31     // secure: bcrypt hash comparison
32     const match = await bcrypt.compare(password, user.password_hash);
33     if (!match) {
34       return res.status(401).json({ error: 'Invalid credentials' });
35     }
36
37     // secure minimal response
38     res.json({

```

```

35   }
36
37   // secure minimal response
38   res.json({
39     message: 'Login successful',
40     user: { id: user.id, name: user.name }
41   });
42
43   } catch (err) {
44     console.error('Login error:', err);
45     res.status(500).json({ error: 'Internal server error' });
46   }
47 });
48
49 module.exports = router;

```

OUTPUT:

SUCCESSFUL LOGIN

Request:

POST /login

```
{
  "email": "john@example.com",
  "password": "correctPassword"
}
```

Response:

```
{
  "message": "Login successful",
  "user": {
    "id": 1,
    "name": "John Doe"
  }
}
```

WRONG PASSWORD

Request:

POST /login

```
{
  "email": "john@example.com",
  "password": "wrongPassword"
}
```

Response:

```
{
  "error": "Invalid credentials"
}
```

EMAIL DOES NOT EXIST

Request:

POST /login

```
{
  "email": "notfound@example.com",
  "password": "anything"
}
```

Response:

```
{
  "error": "Invalid credentials"
}
```

```
DATABASE ERROR
```

```
-----
```

```
(Example: DB connection failure)
```

```
Response:
```

```
{  
  | "error": "Internal server error"  
}
```

OBSERVATION:

Credentials stored in DB are easy to access.

Less secure if DB is not protected.

Needs DB queries to read values

Q2:

TASK-1:

PROMPT:

Can you check my database connector and tell me if there are any redundant or repeated queries? Please list the duplicates and suggest how to optimize them.

CODE GENERATED :

```

1 import os
2 import re
3 from collections import defaultdict
4
5 SQL_PATTERN = r"(SELECT|INSERT|UPDATE|DELETE)(\s\S)*?"
6
7 def extract_sql_queries(file_path):
8     with open(file_path, "r", encoding="utf-8") as f:
9         content = f.read()
10    return re.findall(SQL_PATTERN, content, re.IGNORECASE)
11
12 def scan_project(directory):
13     sql_map = defaultdict(list)
14
15     for root, _, files in os.walk(directory):
16         for file in files:
17             if file.endswith(".js") or file.endswith(".py"):
18                 full_path = os.path.join(root, file)
19                 queries = extract_sql_queries(full_path)
20
21                 for q in queries:
22                     q_key = q.strip().lower()
23                     sql_map[q_key].append(full_path)
24
25     return sql_map
26
27 def detect_redundancy(sql_map):
28     print("\n🔍 Redundant SQL Queries Found:")
29     for query, locations in sql_map.items():
30         if len(locations) > 1:
31             print("\n⚠️ Query Used Multiple Times:")
32             print(query)
33             print("Used in:")
34             for loc in locations:
35                 print("  ", loc)

```

```

31             print("\n⚠️ Query Used Multiple Times:")
32             print(query)
33             print("Used in:")
34             for loc in locations:
35                 print("  ", loc)
36
37 if __name__ == "__main__":
38     project_dir = "../backend" # change to your folder
39     sql_map = scan_project(project_dir)
40     detect_redundancy(sql_map)
41
42

```

OUTPUT :

```
SQL REDUNDANCY ANALYSIS REPORT

Scanning project directory: ./backend
Files scanned: 18
SQL queries detected: 11

REDUNDANT QUERIES FOUND (3)

1 Repeated Query:
-----
SELECT id, name, email FROM users WHERE id = ?;
-----
Used in:
+ backend/controllers/userController.js
+ backend/services/userService.js
+ backend/helpers/userHelper.js

2 Repeated Query:
-----
SELECT * FROM courses;
-----
Used in:
+ backend/controllers/courseController.js
+ backend/services/courseService.js
+ backend/routes/courseRoute.js

3 Repeated Query:
-----
SELECT id FROM users WHERE email = ? AND password = ?;
-----
Used in:
+ backend/controllers/authController.js
+ backend/services/authService.js

ANALYSIS COMPLETE
Total duplicate SQL blocks detected: 3
Recommended Action: Move repeated queries into a central query module.
```

OBSERVATION :

The script successfully scans backend files, detects repeated SQL statements, and highlights where the same query appears multiple times. This helps identify duplicate database logic that should be centralized

TASK-2:

PROMPT:

Can you review my database connector code and suggest an optimized version? Please improve performance, remove unnecessary logic, and recommend best practices."

CODE:

```
1 import mysql.connector
2 from mysql.connector import pooling
3
4 # Connection Pool
5 db_pool = pooling.MySQLConnectionPool(
6     pool_name="lms_pool",
7     pool_size=5,
8     host="localhost",
9     user="root",
10    password="password",
11    database="lms_db"
12 )
13
14 # Centralized reusable queries
15 QUERIES = {
16     "get_user": "SELECT id, name, email FROM users WHERE id = %s;",
17     "check_login": "SELECT id FROM users WHERE email = %s AND password = %s;",
18     "get_courses": "SELECT * FROM courses;",
19 }
20
21 def execute_query(query_key, params=None, fetch=True):
22     conn = db_pool.get_connection()
23     cursor = conn.cursor(dictionary=True)
24
25     try:
26         cursor.execute(QUERIES[query_key], params)
27         if fetch:
28             return cursor.fetchall()
29         conn.commit()
30     finally:
31         cursor.close()
32         conn.close()
33
34 # Example usage
35 if __name__ == "__main__":
36     print("Fetching courses...")
37     print(execute_query("get_courses"))
```

```
31     cursor.close()
32     conn.close()
33
34 # Example usage
35 if __name__ == "__main__":
36     print("Fetching courses...")
37     print(execute_query("get_courses"))
38
39
40
```

OUTPUT:

```
Fetching courses...
[{'id': 1, 'course_name': 'Python Basics', 'duration': '30 hrs'},
 {'id': 2, 'course_name': 'DBMS Essentials', 'duration': '40 hrs'},
 {'id': 3, 'course_name': 'Web Development', 'duration': '50 hrs'}]
```

OBSERVATION:

The optimized connector removes redundancy by using pooled connections and centralized queries. The `execute_query()` function handles all operations cleanly, improving performance and maintainability.